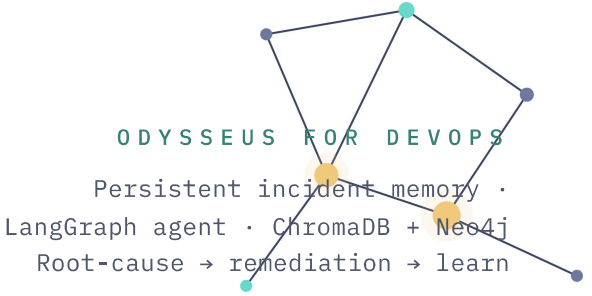
AI-POWERED ENGINEERING OPERATIONS WORKSPACE

AEON.



One for All —
one architecture for every domain.

The first ops workspace that functions as your team's **institutional memory**. It doesn't just show you the error — it remembers how you fixed it last time.

Teams don't lose time to bugs. They lose it to **forgetting.**	**40%** of engineering time spent on toil — re-investigating known failures	**3AM** the on-call reality: digging through months-old logs, half-asleep	**∞** knowledge walks out the door every time an engineer leaves

What Aeon does

remember · diagnose · act · learn

01 — INCIDENT RESPONSE

Diagnose in seconds

A build fails. Aeon streams live tool calls — fetch logs, search memory, query graph — and lands on the root cause with a confidence score.

"Matches Incident #421 · 94% similar · fix attached."

02 — BLAST RADIUS

See what breaks

Before you merge, Aeon maps every changed file to impacted services on a radial graph, then rates deploy risk High / Medium / Low.

"No more 'oops, I broke production.'"

03 — CODE PROVENANCE

Know the why

Ask "why is this file the way it is?" Aeon traces commits → PRs → issues and narrates how the code evolved — the intent behind every line.

"Onboard in days, not quarters."

04 — HUMAN-IN-THE-LOOP

Approve, then act

Aeon auto-creates issues and proposes PRs — but never ships to production without a human clicking Approve. Safety as a design principle.

"It empowers engineers, it doesn't replace them."

Aeon gives your team an **AI-powered persistent memory** — so you never solve the same incident twice, and you see the risk of every change before it ships.

NEVER SOLVE THE SAME BUG TWICE

ARCHITECTURE & RETURN ON INVESTMENT

Built like a platform. Priced like a payback.

One agentic core — GitHub, Jenkins & n8n in, memory that compounds out.

FastAPI · LangGraph
Claude Sonnet 4.6 · 8 tools
ChromaDB + Neo4j

System Architecture

SSE streaming end-to-end

Browser — React · Vite · Tailwind

↓

FastAPI Backend — :8000 · per-token SSE

↓

LangGraph Agent — Claude Sonnet 4.6 · astream()

↓

ChromaDB vector recall — Neo4j graph memory

↓ integrations

GitHub — Jenkins — n8n

```
search_memory → call_claude → execute_tools (loop ≤15×) → synthesize → memory_writer
every analysis is written back — the agent gets smarter with each run.
```

Stack

8 services · Docker Compose

FastAPI · Python 3.12 | LangGraph StateGraph | Claude sonnet-4-6 | ChromaDB | Neo4j | PostgreSQL | Redis | React 18 + Vite | react-force-graph-2d | Jenkins + GitHub Actions | n8n

The Numbers

per 20-engineer team

60%
reduction in Mean Time to Resolution (MTTR)

$54K
engineer-time recovered per year

WHERE THE TIME GOES	BEFORE → AFTER	MONTHLY GAIN
Root-cause per incident	90m → **30m**	40 hrs
Repeat incidents (auto-matched)	re-solved → **recalled**	20 hrs
Bad deploys avoided (Blast Radius)	guess → **risk score**	10 hrs
≈ 70 hrs/month × $75/hr loaded		**$4.5K**

Illustrative model — 20 engineers, ~40 incidents/month, $75/hr loaded cost. Scales linearly with team size; excludes reduced burnout, faster onboarding & cloud (FinOps) savings.

< 1 mo
Payback vs. a single senior engineer's time.
ROI compounds — memory only gets richer with use.